# Causal World Models: Object-Centric Directed Interaction Learning and Constant-Depth Latent State Simulation

Artem Zhidkikh
*Preliminary Technical Report*

**Abstract**

Autoregressive sequence models represent a future as a succession of token-level conditional distributions. This representation is effective for language modeling, but it is an indirect and computationally serial interface for predicting a structured world transition: the model must repeatedly describe the future rather than update a state of the world. We introduce the *Causal World Model* (CWM), a compact object-centric architecture that maps a tokenized observation into a fixed set of latent object slots, constructs a dense directed interaction tensor between slots, and advances the entire latent configuration through one residual dynamics step. The model is trained without next-token likelihood. A shared encoder embeds an observed successor descriptor, and a latent energy measures the discrepancy between the predicted and observed object configurations.

CWM replaces token–token self-attention, whose dense attention term is quadratic in input length, with linear-in-length token-to-slot aggregation followed by quadratic interaction only over a small, fixed object budget. A single future transition is computed with constant sequential depth and with one model invocation, rather than one invocation per generated target token. Thus, for fixed slot count and latent width, latent forecasting has $\mathcal{O}(1)$ sequential decoding depth with respect to the length of the target serialization and requires no growing key–value cache. This statement does *not* imply $\mathcal{O}(1)$ arithmetic complexity: reading the input is at least linear in its length, and the dense object graph remains quadratic in the number of slots.

To prevent an identity-transition shortcut, we further construct a 10,000-example high-dynamics corpus containing six families of irreversible events. Each target is a compact state delta rather than a near-copy of the initial scene, and a structural validator rejects unchanged assignments, static scene fragments, weak ASCII motion cues, and targets whose character-length ratio exceeds 0.78. In a preliminary single-run experiment on an NVIDIA Tesla T4 GPU, the reported training energy decreased from $6.0 \times 10^{-3}$ to $4.0 \times 10^{-6}$ over 30 epochs. In a held-out three-way transition-ranking test, a directed destructive outcome received energy $1.01 \times 10^{-4}$, compared with $1.37 \times 10^{-4}$ for static continuation and $3.27 \times 10^{-4}$ for unstructured chaos. These results show that the learned energy ranks the intended dynamic continuation above an identity hypothesis on the reported synthetic test. They are an initial proof of concept, not yet evidence of causal identification or general-purpose physical reasoning.

## 1 Introduction

The dominant computational metaphor in contemporary language modeling is autoregression. Given a prefix $x_{1:t}$, a model estimates $p(x_{t+1} \mid x_{1:t})$, appends a sampled or selected token, and repeats. The factorization

$$p(x_{1:K}) = \prod_{k=1}^{K} p(x_k \mid x_{<k}) \tag{1}$$

is statistically general and has supported remarkable empirical progress. The Transformer makes this factorization highly parallel during teacher-forced training through self-attention [1]. Nevertheless, the factorization imposes a serial interface at inference: a target of $K$ tokens requires $K$ dependent decoding decisions. A key–value (KV) cache avoids recomputing all past keys and values, but cache memory grows with the processed context, layers, and attention heads; each new token must still attend to an expanding history. Modern systems engineering can substantially reduce the input/output cost of attention [2, 3], but it does not change the semantic fact that the future is emitted token by token.

This interface is natural when the desired object is text. It is less natural when the desired object is a transition of a structured environment. Consider a fragile vase pushed from a table. The physically salient content is a coupled state change: support vanishes, vertical velocity becomes negative, integrity reaches zero, and debris appears. An autoregressive model must express these consequences as an ordered string, even though the underlying transition is simultaneous at the chosen temporal resolution. Token likelihood also rewards predictable syntax, tags, and unchanged descriptions. When most of a serialized successor repeats the initial scene, a latent predictor can exploit an analogous shortcut: it can approximate the identity map and achieve low average error without learning dynamics.

The scaling of model size and data has repeatedly improved language-model quality [4]. We do not claim that this scaling program is literally at an algorithmic dead end. We instead argue that scaling alone does not remove an architectural mismatch between token continuation and world-state simulation. Larger autoregressive models remain autoregressive; longer contexts increase state retention requirements; and a richer textual description of a physical event is not the same object as the event's transition operator. This motivates a complementary direction: learn a compact state, infer interactions among its entities, and predict the next state in one parallel latent update.

We propose CWM as a small-footprint realization of this idea. Its principal inductive biases are:

1. **Object-centric coarse graining.** A token sequence is compressed into $N$ learned macro-object slots rather than processed by token–token global attention.

2. **Directed relational structure.** Ordered source and target projections parameterize a distinct vector for every potential influence $i \to j$.

3. **Residual latent dynamics.** Incoming and outgoing relational effects are integrated into an object-level force, and a multilayer transition predicts a state increment.

4. **Energy-based compatibility.** The predicted successor and an encoded observed successor receive a scalar discrepancy that can be minimized directly and used to rank candidate futures.

5. **High-dynamics supervision.** Synthetic targets contain only the changed elements of an event, eliminating most lexical incentive for a frozen transition.

The resulting reference configuration uses approximately one million trainable parameters and operates on fixed-size tensors. It is intended as an architectural prototype for low-resource state prediction, not as a drop-in language model: the current system has no free-form text decoder, does not establish a structural causal model in the interventionist sense, and has so far been evaluated only on synthetic single-step transitions. These boundaries are central to the scientific claim.

**Contributions.** The paper makes four concrete contributions. First, it specifies an end-to-end object-centric latent transition model whose equations correspond directly to a working PyTorch implementation. Second, it gives a careful complexity analysis that distinguishes constant sequential transition depth from arithmetic $\mathcal{O}(1)$. Third, it introduces a deterministic high-dynamics dataset generator with explicit anti-static validation. Fourth, it reports preliminary optimization and out-of-distribution (OOD) candidate-ranking measurements on a Tesla T4, together with the limitations necessary to interpret them responsibly.

# 2 Related Work

## 2.1 Transformers, Efficient Attention, and KV Caching

Dense self-attention computes pairwise interactions among $T$ token positions and therefore contains a $\Theta(T^2D)$ term for latent width $D$ [1]. FlashAttention reduces memory traffic while preserving exact attention [2]; other inference-oriented methods reduce the bandwidth cost of retrieving cached histories [3]. These methods are complementary to CWM. They optimize token-centric inference, whereas CWM changes the prediction target from a token sequence to a fixed-size latent state. If a future CWM system adds an autoregressive language decoder, the decoder will again inherit autoregressive costs; the present efficiency claim applies to latent state prediction itself.

## 2.2 Object-Centric Representation Learning

Object-centric learning seeks representations that factor a scene into a set of entities. Slot Attention introduced an iterative competitive mechanism that maps perceptual features to abstract slots [5]. CWM uses a lighter single-pass token-to-slot assignment with learned slot prototypes and an importance gate. Unlike the randomized, exchangeable slots often used for visual object discovery, CWM's learned prototypes also provide a persistent slot reference across the two sides of a transition. This persistence makes direct slot-wise regression possible, although it does not guarantee semantic identity and may fail when interchangeable entities swap slots.

## 2.3 Graph Networks and Structured World Models

Graph networks supply a relational inductive bias by representing entities as nodes and interactions as edges [6]. Neural Relational Inference jointly learns latent relations and dynamics from observational trajectories [7]. Contrastively trained Structured World Models combine object-oriented representations with graph transitions [8]. CWM shares the hypothesis that factorized entities and learned relations can support compositional dynamics, but differs in its input modality, single-pass coarse graining, explicit ordered source/target edge features, and delta-centered textual supervision.

## 2.4 World Models and Energy-Based Learning

World models learn compressed temporal representations that can support prediction, control, or planning [9]. Energy-based models assign a scalar compatibility to configurations rather than requiring a normalized probability [10]. Recent proposals for autonomous machine intelligence have likewise emphasized predictive latent representations and energy-based objectives [11]. The present critic is deliberately minimal: it is a positive- pair latent mean-squared error (MSE), or optional cosine distance, interpreted as an energy. Because it does not yet use explicit negative samples, a partition function, or a target-network anti-collapse mechanism, it should be understood as an energy-shaped regression objective rather than a complete contrastive EBM training scheme.

## 3 Problem Formulation

Let $x^{(0)} \in \mathbb{N}^{T_0}$ be a tokenized observation of a world at time $t$ and let $x^{(1)} \in \mathbb{N}^{T_1}$ be an observed successor descriptor. In the high-dynamics training regime used here, $x^{(0)}$ is a full scene serialization and $x^{(1)}$ is a compact event delta containing only changed state. We seek a parameterized map

$$F_\theta : x^{(0)} \mapsto \widehat{\boldsymbol{Z}}^{(1)} \in \mathbb{R}^{N \times D}, \tag{2}$$

where $N$ is a fixed object-slot budget and $D$ is the dimension of each object vector. A shared encoder produces the target

$$\boldsymbol{Z}^{(1)} = \mathrm{Enc}_\theta(x^{(1)}) \in \mathbb{R}^{N \times D}. \tag{3}$$

The training objective lowers the energy of the observed transition:

$$\min_\theta \; \mathbb{E}_{(x^{(0)}, x^{(1)}) \sim \mathcal{D}} \left[ E_\theta \Big( F_\theta(x^{(0)}), \mathrm{Enc}_\theta(x^{(1)}) \Big) \right]. \tag{4}$$

No next-token distribution appears in Equation (4). The system predicts all $N$ target vectors concurrently.

The term "causal" denotes the architectural hypothesis that ordered, object-level interactions are useful for state transition. From observational synthetic pairs alone, directed attention weights are not causally identifiable in the formal sense: there is no learned directed acyclic graph constraint, interventional calculus, or proof that an edge equals an intervention effect. We therefore use *directed interaction graph* for the mathematical object and reserve *causal graph* for the intended semantics to be tested in future interventional experiments.

## 4 Architecture Components

### 4.1 Notation and Tensor Flow

For a batch of size $B$, padded token IDs are denoted $\boldsymbol{X} \in \mathbb{N}^{B \times T}$ and their validity mask by $\boldsymbol{M} \in \{0, 1\}^{B \times T}$. The principal tensors are

$$\boldsymbol{X} \longrightarrow \boldsymbol{H} \in \mathbb{R}^{B \times T \times D} \longrightarrow \boldsymbol{Z} \in \mathbb{R}^{B \times N \times D}, \tag{5}$$

$$\boldsymbol{Z} \longrightarrow \boldsymbol{E} \in \mathbb{R}^{B \times N \times N \times E} \longrightarrow \boldsymbol{Z}' \in \mathbb{R}^{B \times N \times D}, \tag{6}$$

$$(\boldsymbol{Z}', \boldsymbol{E}) \longrightarrow \widehat{\boldsymbol{Z}}^{(1)} \in \mathbb{R}^{B \times N \times D}. \tag{7}$$

Here $E$ is the edge-vector width. We suppress the batch index when it does not affect the derivation.

### 4.2 CWMObjectEncoder: Macro-Object Coarse Graining

#### 4.2.1 Token and local-context representation

Each input token receives a content embedding and an absolute positional embedding:

$$\boldsymbol{r}_t = \boldsymbol{W}_{\mathrm{tok}}[x_t] + \boldsymbol{W}_{\mathrm{pos}}[t] \in \mathbb{R}^D. \tag{8}$$

A depthwise one-dimensional convolution of kernel width five extracts local context, and a pointwise convolution mixes feature channels. With $\mathcal{C}_{\mathrm{dw}}$ and $\mathcal{C}_{\mathrm{pw}}$ denoting these operators,

$$\boldsymbol{\ell}_{1:T} = \mathrm{Dropout}\big(\mathrm{GELU}\big(\mathcal{C}_{\mathrm{pw}}\big(\mathcal{C}_{\mathrm{dw}}(\boldsymbol{r}_{1:T})\big)\big)\big), \tag{9}$$

and

$$\boldsymbol{h}_t = m_t \,\mathrm{LN}(\boldsymbol{r}_t + \boldsymbol{\ell}_t). \tag{10}$$

This operation is a local coarse-graining stage: it combines adjacent lexical cues without constructing a $T \times T$ token-attention matrix.

#### 4.2.2 Competitive token-to-slot assignment

The encoder maintains $N$ learned object prototypes $\boldsymbol{q}_n \in \mathbb{R}^D$. Token features are projected to keys and values,

$$\boldsymbol{k}_t = \boldsymbol{W}_K \boldsymbol{h}_t, \qquad \boldsymbol{v}_t = \boldsymbol{W}_V \boldsymbol{h}_t, \tag{11}$$

and every valid token competes across slots:

$$a_{tn} = \frac{\boldsymbol{k}_t^\top \boldsymbol{q}_n}{\sqrt{D}}, \qquad \pi_{tn} = \frac{\exp(a_{tn})}{\sum_{r=1}^N \exp(a_{tr})}. \tag{12}$$

The softmax in Equation (12) is taken over object slots, not over tokens. Consequently, each token distributes its explanatory mass among a fixed set of macro-objects.

#### 4.2.3 Importance gate

Not every token should contribute equally to a world state. Formatting characters, articles, and recurring tags can be syntactically common but dynamically irrelevant. The scalar importance gate is

$$g_t = \mathrm{sigmoid}(\boldsymbol{w}_g^\top \boldsymbol{h}_t + b_g), \qquad g_t \in (0, 1). \tag{13}$$

After gating, weights are normalized across the token axis separately for each slot:

$$\omega_{tn} = \frac{m_t g_t \pi_{tn}}{\max\left(\epsilon, \sum_{u=1}^T m_u g_u \pi_{un}\right)}, \qquad \epsilon = 10^{-6}. \tag{14}$$

This two-stage normalization has a useful interpretation. The slot softmax makes objects compete for each token; the token normalization then makes the selected evidence form a convex aggregation for each object, up to the numerical clamp.

The initial slot state is

$$\widetilde{\boldsymbol{z}}_n = \boldsymbol{q}_n + \sum_{t=1}^T \omega_{tn} \boldsymbol{v}_t. \tag{15}$$

A residual object MLP refines the pooled representation:

$$\boldsymbol{z}_n = \mathrm{LN}\big(\widetilde{\boldsymbol{z}}_n + \mathrm{MLP}_{\mathrm{obj}}\big(\mathrm{LN}(\widetilde{\boldsymbol{z}}_n)\big)\big)\,. \tag{16}$$

The same learned layer-normalization module is reused before the MLP and at the final output in the reference implementation. The result $\boldsymbol{Z} = [\boldsymbol{z}_1, \ldots, \boldsymbol{z}_N]$ has a fixed shape independent of input length.

## 4.3 CausalGraphLayer: Asymmetric Directed Interactions

### 4.3.1 Ordered pair construction

For every object $i$, separate affine maps construct a source representation and a target representation:

$$\boldsymbol{s}_i = \boldsymbol{W}_s \boldsymbol{z}_i + \boldsymbol{b}_s, \qquad \boldsymbol{t}_i = \boldsymbol{W}_t \boldsymbol{z}_i + \boldsymbol{b}_t, \qquad \boldsymbol{s}_i, \boldsymbol{t}_i \in \mathbb{R}^P. \tag{17}$$

For the ordered pair $i \to j$, the graph layer forms

$$\boldsymbol{p}_{ij} = [\boldsymbol{s}_i; \boldsymbol{t}_j; \boldsymbol{s}_i - \boldsymbol{t}_j; \boldsymbol{s}_i \odot \boldsymbol{t}_j] \in \mathbb{R}^{4P}, \tag{18}$$

where $[\cdot;\cdot]$ denotes concatenation and $\odot$ elementwise multiplication. The edge vector is

$$\boldsymbol{e}_{ij} = \mathrm{LN}_E(\mathrm{MLP}_{\mathrm{edge}}(\boldsymbol{p}_{ij})) \in \mathbb{R}^E. \tag{19}$$

Because $\boldsymbol{W}_s$ and $\boldsymbol{W}_t$ are distinct and the concatenation order is fixed, the parameterization permits $\boldsymbol{e}_{ij} \neq \boldsymbol{e}_{ji}$. Directionality is enabled rather than mathematically forced: a trained parameter setting can still become approximately symmetric.

### 4.3.2 Edge-conditioned object update

An edge score determines how strongly source $i$ contributes to target $j$:

$$\alpha_{ij} = \mathrm{softmax}_i\left(\frac{\boldsymbol{w}_e^\top \boldsymbol{e}_{ij} + b_e}{\sqrt{E}}\right). \tag{20}$$

The normalization runs over all possible sources for each fixed target. Pairwise messages combine the source object and edge content,

$$\boldsymbol{m}_{ij} = \boldsymbol{W}_m \boldsymbol{z}_i + \boldsymbol{W}_{em} \boldsymbol{e}_{ij}, \tag{21}$$

and the target receives

$$\boldsymbol{\mu}_j = \sum_{i=1}^{N} \alpha_{ij} \boldsymbol{m}_{ij}. \tag{22}$$

The graph-refined object is

$$\boldsymbol{z}'_j = \mathrm{LN}_D(\boldsymbol{z}_j + \mathrm{MLP}_{\mathrm{update}}([\boldsymbol{z}_j; \boldsymbol{\mu}_j])). \tag{23}$$

The graph layer includes diagonal edges during this contextual update. The physics engine, described next, removes diagonal edges when computing external forces.

## 4.4 LatentPhysicsEngine: Residual State Integration

The physics engine maps each edge into object-state coordinates:

$$\boldsymbol{r}_{ij} = \boldsymbol{W}_p \boldsymbol{e}_{ij} + \boldsymbol{b}_p \in \mathbb{R}^D. \tag{24}$$

Let $\eta_N = \max(N-1, 1)$. Self-interactions are masked, and outgoing and incoming effects are averaged separately:

$$\boldsymbol{o}_i = \frac{1}{\eta_N} \sum_{j \neq i} \boldsymbol{r}_{ij}, \tag{25}$$

$$\boldsymbol{u}_i = \frac{1}{\eta_N} \sum_{j \neq i} \boldsymbol{r}_{ji}. \tag{26}$$

Although both terms are formed from the same projected edge tensor, their ordering is retained by concatenation:

$$\boldsymbol{f}_i = \mathrm{MLP}_{\mathrm{fusion}}([\boldsymbol{o}_i; \boldsymbol{u}_i]) \in \mathbb{R}^D. \tag{27}$$

The transition network plays the role of a learned discrete differential operator:

$$\Delta \boldsymbol{z}_i = \mathrm{MLP}_{\mathrm{physics}}\big([\mathrm{LN}(\boldsymbol{z}'_i); \mathrm{LN}(\boldsymbol{f}_i)]\big)\,. \tag{28}$$

One Euler-like residual step produces the predicted successor,

$$\widehat{\boldsymbol{z}}_i^{(1)} = \boldsymbol{z}'_i + \Delta \boldsymbol{z}_i. \tag{29}$$

Equation (29) provides a direct gradient path and allows the network to learn small local updates when the underlying process is smooth. The final linear layer of $\mathrm{MLP}_{\mathrm{physics}}$ is initialized from $\mathcal{N}(0, 10^{-6})$, equivalently a weight standard deviation of $10^{-3}$, and its bias is initialized to zero. This starts the system near an identity transition without making the initial Jacobian exactly zero.

Residual dynamics and layer normalization improve numerical conditioning, but they do not prove non-collapse. Indeed, the near-identity initialization makes a frozen solution initially accessible. The high-dynamics target distribution is therefore not a cosmetic data choice; it supplies the gradient pressure needed to move away from that basin.

### 4.5 CWMCritic: Latent Surprise Energy

Let $\widehat{\boldsymbol{Z}}^{(1)}$ be the prediction and $\boldsymbol{Z}^{(1)} = \mathrm{Enc}_\theta(x^{(1)})$ the target. The default per-object energy is

$$\varepsilon_{bn}^{\mathrm{MSE}} = \frac{1}{D}\sum_{d=1}^{D}\left(\widehat{Z}_{bnd}^{(1)} - Z_{bnd}^{(1)}\right)^2. \tag{30}$$

The scalar loss is

$$\mathcal{L}_{\mathrm{CWM}} = \frac{1}{BN}\sum_{b=1}^{B}\sum_{n=1}^{N}\varepsilon_{bn}^{\mathrm{MSE}} = \frac{1}{BND}\left\|\widehat{\boldsymbol{Z}}^{(1)} - \boldsymbol{Z}^{(1)}\right\|_F^2. \tag{31}$$

An optional cosine energy is

$$\varepsilon_{bn}^{\cos} = 1 - \frac{\widehat{\boldsymbol{z}}_{bn}^{(1)\top}\boldsymbol{z}_{bn}^{(1)}}{\|\widehat{\boldsymbol{z}}_{bn}^{(1)}\|_2\|\boldsymbol{z}_{bn}^{(1)}\|_2 + \epsilon}. \tag{32}$$

At evaluation time, candidate futures $y \in \mathcal{H}$ can be ranked by

$$y^* = \arg\min_{y\in\mathcal{H}} E_\theta\Big(F_\theta(x^{(0)}), \mathrm{Enc}_\theta(y)\Big)\,. \tag{33}$$

This is an energy-based decision rule: no normalized likelihood is required. Nevertheless, the current positive-pair objective admits a degenerate constant encoder. The shared target encoder is neither stopped-gradient nor exponentially averaged. The OOD ranking reported later is evidence against complete collapse on that test, not a theorem excluding collapse in general.

### 4.6 End-to-End Forward Pass

The full computation for a pair $(x^{(0)}, x^{(1)})$ is

$$\begin{aligned}
\boldsymbol{Z}^{(0)} &= \mathrm{Enc}_\theta(x^{(0)}), && (34)\\
(\boldsymbol{Z}'^{(0)}, \boldsymbol{E}^{(0)}) &= \mathrm{Graph}_\theta(\boldsymbol{Z}^{(0)}), && (35)\\
\widehat{\boldsymbol{Z}}^{(1)} &= \mathrm{Phys}_\theta(\boldsymbol{Z}'^{(0)}, \boldsymbol{E}^{(0)}), && (36)\\
\boldsymbol{Z}^{(1)} &= \mathrm{Enc}_\theta(x^{(1)}), && (37)\\
\mathcal{L} &= \mathrm{Critic}\left(\widehat{\boldsymbol{Z}}^{(1)}, \boldsymbol{Z}^{(1)}\right). && (38)
\end{aligned}$$

Automatic differentiation propagates through the physics engine, graph layer, and both uses of the object encoder. Dropout in the encoder and graph layer makes training-time computation stochastic. Conditional on fixed parameters and disabled dropout, the latent transition itself is deterministic.

### 4.7 Computational Complexity and the Meaning of Constant-Time Inference

Let $P$ be the graph projection width, $H_g$ the edge-MLP width, and $H_p$ the physics-MLP width. Ignoring batch size and constants, the major costs are

$$\begin{aligned}
C_{\text{enc}} &= \mathcal{O}\big(TD^2 + TND + ND^2\big), && (39)\\
C_{\text{graph}} &= \mathcal{O}\big(NDP + N^2(PH_g + H_gE + ED) + ND^2\big), && (40)\\
C_{\text{physics}} &= \mathcal{O}\big(N^2ED + ND^2 + NDH_p + H_p^2N\big). && (41)
\end{aligned}$$

For latent widths proportional to $D$, this simplifies to

$$C_{\text{CWM}} = \mathcal{O}\big(TD^2 + TND + N^2D^2 + ND^2\big). \quad (42)$$

Memory includes $\mathcal{O}(TD + TN + N^2E)$. In contrast, dense Transformer self-attention contains a $\mathcal{O}(T^2D)$ attention term and stores a context-dependent KV state during autoregressive inference.

The CWM claim is therefore precise:

> For one-step prediction, CWM uses one constant-depth latent transition independent of the number of textual tokens that would be needed to describe the future. Once an encoded state and a fixed $N, D, E$ configuration are given, the transition has fixed cost and may be called $\mathcal{O}(1)$ with respect to target serialization length. It is not $\mathcal{O}(1)$ with respect to input length, object count, latent width, or rollout horizon.

An $H$-step rollout costs $\mathcal{O}(H)$ applications of the physics engine. A variable object budget retains the $N^2$ graph term. Shorter strings also do not reduce the current padded implementation's GPU arithmetic unless padding-aware kernels or length bucketing are introduced.

# 5 Synthetic High-Dynamics Dataset

### 5.1 Motivation: The Latent Freezing Shortcut

Suppose a serialized successor repeats most of the initial scene. If the encoder is locally smooth, then

$$\mathrm{Enc}(x^{(1)}) \approx \mathrm{Enc}(x^{(0)}) + \boldsymbol{\epsilon}, \qquad \|\boldsymbol{\epsilon}\| \ll 1. \quad (43)$$

A residual predictor initialized near the identity can obtain low error with $\Delta \boldsymbol{Z} \approx \mathbf{0}$. The gradient of Equation (31) with respect to a predicted state increment is

$$\frac{\partial \mathcal{L}}{\partial \Delta \boldsymbol{Z}} = \frac{2}{BND} \left( \boldsymbol{Z}'^{(0)} + \Delta \boldsymbol{Z} - \boldsymbol{Z}^{(1)} \right). \tag{44}$$

When the target embedding is almost unchanged, the force moving the transition away from zero is correspondingly weak. This optimization pathology is called *latent freezing* in this work.

The revised dataset changes the supervision interface. The input $x^{(0)}$ remains a complete description; the target $x^{(1)}$ is an event-centered delta containing only changed variables, terminal event language, and displaced or destroyed ASCII entities. The target is therefore a *transition-sufficient successor descriptor*, not a complete reconstruction of the world. This increases representation-level contrast, but also creates a full-state/delta distribution mismatch that must be addressed in future real-world systems.

## 5.2 Serialization

Every sample is a single line with three sections:

$$\begin{aligned} x = {} & \texttt{[TEXT\_PHYSICS]}: \text{event text} \quad \texttt{[CODE\_LOGIC]}: \text{state assignments} \\ & \texttt{[ASCII\_MAP]}: \text{spatial symbols}. \end{aligned} \tag{45}$$

The same outer tags make the modality boundaries explicit, while the content of T1 is deliberately terse. Character-level tokenization builds a corpus vocabulary with padding index zero and unknown index one. In the current generated corpus the vocabulary contains 114 entries and all strings fit within a 512-character limit.

## 5.3 Six Destructive Transition Families

The generator creates 10,000 examples by cycling through six scenario builders, with seeded randomization of counts, coordinates, velocities, payloads, and damage quantities. The first four families contain 1,667 examples each and the final two 1,666 each.

The target text begins with a scenario-specific marker such as `*CRASH_DESTRUCTION*`, `*CHAIN_COLLAPSE*`, `*SLOT_OVERWRITE*`, `*GRAVITY_COLLAPSE*`, `*TANK_EXPLOSION*`, or `*COLLISION_CASCADE*`. ASCII targets use high-amplitude visual cues, including `-->`, `*Crash*`, `[Broken_X]`, `[FALLING_X]`, `[EXPLOSION]`, `[OVERWRITE]`, and `[CASCADE]`.

## 5.4 Static-Transition Rejection

The validator parses all three sections and applies the following rules before an example enters the dataset:

1. T0 and T1 must differ in physics text, code, and ASCII map.
2. T1 physics must begin with one of the six aggressive dynamic markers.
3. T1 must contain at least three assignments. Every assigned variable must already exist in T0, and every assigned value must differ from its T0 value.
4. T1 ASCII must contain an explicit motion or destruction marker.
5. T1 may not contain any member of `FORBIDDEN_T1_FRAGMENTS`: `scene_id`, `[TABLE`, `[Robot`, `[Pump`, `[Hand`, `[Support`, or the unchanged assignment `limit =`.

Table 1: High-dynamics transition families. Every target changes code variables and redraws the spatial state.

| Family | Intervention | Required T1 delta |
|---|---|---|
| Vase destruction | A cat strikes a fragile vase at the table edge. | Integrity is zero, debris and broken-item counts increase, and `*Crash* [Broken_Vase]` replaces the intact object. |
| Domino reaction | The first block receives a directed impulse. | Every block reaches a fallen angle and every serialized coordinate changes; standing count becomes zero. |
| While-loop overwrite | One loop iteration reads a payload from a source slot. | The counter decreases, the source is cleared, the active slot changes, and a rewritten payload appears in the target slot. |
| Tower collapse | A robot removes the bottom support cube. | Stable count becomes zero, all cubes acquire downward states and new coordinates, and vertical speed is negative. |
| Tank explosion | Pressure exceeds the casing limit of a closed tank. | Pressure and integrity become zero; fragments and blast radius become positive; only the explosion and broken tank remain in T1. |
| Collision cascade | A fast heavy ball strikes a row of crates. | Every crate receives motion and a new position; collision and moving-object counters increase. |

6. The serialized-length ratio must satisfy

$$\rho(x^{(0)}, x^{(1)}) = \frac{|x^{(1)}|}{|x^{(0)}|} \leq \rho_{\max}, \qquad \rho_{\max} = 0.78. \tag{46}$$

Across the generated 10,000 examples, the observed maximum ratio is approximately 0.696, the mean ratio approximately 0.663, and the maximum T1 length 248 characters. No generated T1 contains `scene_id`.

These checks are syntactic safeguards, not a physics engine. The blacklist cannot prove that all unchanged information has been removed, the length ratio is not a semantic-overlap measure, and hand-written templates do not establish physical validity outside their construction rules. Explicit markers may themselves become shortcuts. A rigorous evaluation must therefore use format-matched counterfactuals in which marker distributions are controlled independently of event correctness.

# 6 Experimental Results

## 6.1 Reference Configuration

The reference architecture uses six object slots, object width $D = 128$, edge width $E = 64$, graph projection width $P = 128$, maximum character sequence length 512, and the 114-entry vocabulary induced by the current corpus. This instantiation contains 995,586 trainable parameters: 197,505 in the object encoder, 230,657 in the graph layer, and 567,424 in the physics engine; the MSE critic has

Table 2: Observed endpoints of the reported Tesla T4 training run.

| Measurement | Training stage | Latent energy loss |
|---|---|---|
| Initial endpoint | Beginning of optimization | 0.006000 |
| Final endpoint | Epoch 30 | 0.000004 |

Table 3: Reported OOD candidate energies. The margin is measured relative to the directed destruction candidate. Lower is better.

| Candidate | Energy | Rank | Absolute margin |
|---|---|---|---|
| Directed destruction | 0.000101 | 1 | 0 |
| Static continuation | 0.000137 | 2 | 0.000036 |
| Chaotic continuation | 0.000327 | 3 | 0.000226 |

no trainable parameters. The parameter count is approximately 3.8 MiB using 32-bit floating-point weights before optimizer state and activations.

Training was performed on an NVIDIA Tesla T4 GPU for 30 epochs. The supplied result record provides loss endpoints and OOD candidate energies but does not provide repeated seeds, the exact train/validation split, per-epoch values, batch size, learning-rate schedule, wall-clock time, peak memory, or the number of OOD trials. We therefore report only the observed values and do not fabricate uncertainty estimates or intermediate points.

### 6.2 Optimization Trace

The scalar training energy decreased from 0.006 at the beginning of optimization to 0.000004 at epoch 30 (Table 2). The ratio of the two reported endpoints is 1,500, corresponding to a 99.933% reduction in the objective. Because the target encoder is jointly optimized, this number alone cannot distinguish improved transition modeling from representational contraction. Candidate ranking and future representation diagnostics are needed to interpret it.

### 6.3 Grand Transfer Examination

The held-out "Grand Exam" presents three candidate successor descriptions for a dynamic initial scene:

1. a structured destructive continuation consistent with the intervention;
2. a static continuation that leaves the world unchanged; and
3. an unstructured chaotic perturbation.

Each candidate is encoded and scored according to Equation (33). Lower energy denotes greater compatibility with the predicted latent successor.

The observed ordering is

$$E_{\text{destruction}} < E_{\text{static}} < E_{\text{chaos}}. \tag{47}$$

The static candidate has approximately 1.36× the energy of the destructive candidate, while chaos has approximately 3.24× its energy. Equivalently, the destructive energy is about 26.3% below the static energy and 69.1% below the chaotic energy. This ranking is the central empirical observation:

the identity hypothesis is no longer the minimum-energy answer, yet arbitrary change remains less compatible than structured destruction.

The absolute scale of latent MSE is representation-dependent. Equation (47) does not yield a calibrated probability, demonstrate statistical significance, or establish general causal reasoning. In particular, a full-state static candidate may differ from the compact delta target in formatting as well as semantics. The present result should be read as a successful preliminary ranking test under the reported protocol.

### 6.4 What the Experiment Establishes

The reported run supports three limited conclusions:

1. The complete architecture is differentiable and can be optimized to very low positive-pair latent energy on the generated corpus.
2. Delta-centered supervision is compatible with escaping the observed identity-transition basin in this implementation.
3. On the reported OOD comparison, the learned energy ranks a structured dynamic continuation ahead of both stasis and unstructured perturbation.

It does not yet establish superiority to Transformers, recurrent networks, state-space models, graph-only baselines, or identity predictors, because no controlled baseline table has been supplied. It also does not establish efficiency in wall-clock terms; the complexity advantage remains an architectural analysis until latency, throughput, and memory are measured under matched conditions.

## 7 Discussion and Future Work

### 7.1 From Character Tokens to Semantic Units

Character tokenization makes the prototype self-contained and guarantees coverage of prose, code, and ASCII symbols. It is nevertheless a poor interface for a production world model. Semantically atomic units such as `vase_integrity`, `counter -= 1`, and `[Broken_Vase]` are fragmented into long character sequences. This increases padding, forces the encoder to relearn lexical composition, and encourages sensitivity to orthographic regularities.

The immediate minimum viable product path is a hybrid tokenizer. Byte-pair encoding or a unigram model can compress natural-language fragments; a code-aware lexer can preserve identifiers, operators, numeric literals, and delimiters; and ASCII entities can be retained as atomic spatial tokens. Modality or segment embeddings should mark physics prose, program state, and spatial maps while allowing a shared object encoder to integrate them. Vocabulary construction must be fitted on the training split only to avoid test-corpus leakage.

For real code, the dataset should move beyond strings to instrumented program transitions. A state can include an abstract syntax tree, variable store, heap graph, control-flow location, test inputs, and execution trace. The successor target should be a structured diff produced by actual execution rather than a template. Splits must hold out algorithms and program templates, not merely identifiers or numeric values. Counterfactual tests should change a single input variable while preserving the program, measuring whether directed edges follow intervention effects.

### 7.2 A Stronger Anti-Collapse Energy

Positive-pair MSE cannot, by itself, exclude a constant representation. A natural extension compares the true future to format-matched negative futures. Let $\mathcal{N}(x^{(0)})$ contain static, shuffled, causally reversed, and dynamically implausible candidates. A margin-ranking objective is

$$\mathcal{L}_{\text{rank}} = \sum_{y^- \in \mathcal{N}(x^{(0)})} \max\Big(0, m + E(\widehat{\boldsymbol{Z}}^{(1)}, \text{Enc}(y^+)) - E(\widehat{\boldsymbol{Z}}^{(1)}, \text{Enc}(y^-))\Big), \tag{48}$$

where $m > 0$ is a margin. Negatives must share tags, length statistics, event markers, and object vocabulary with the positive target; otherwise the model can solve the task through formatting.

Additional safeguards include an exponentially averaged target encoder, stop-gradient targets, and explicit representation variance/covariance regularization. Publication- grade diagnostics should report slot-wise variance, covariance spectrum, effective rank, sensitivity to input perturbations, and energy under shuffled object identities.

### 7.3 Permutation and Object Identity

Direct MSE assumes stable correspondence between predicted and target slots. Learned prototypes encourage alignment, but scenes with interchangeable objects can exchange slot identities without changing physical meaning. A permutation-aware objective can replace Equation (31) with

$$\mathcal{L}_{\text{set}} = \min_{\pi \in S_N} \frac{1}{ND} \sum_{n=1}^{N} \left\| \widehat{\boldsymbol{z}}_n^{(1)} - \boldsymbol{z}_{\pi(n)}^{(1)} \right\|_2^2, \tag{49}$$

where $S_N$ is the permutation group. Hungarian matching, differentiable optimal transport, or explicit persistent object IDs are possible implementations.

### 7.4 Sparse Relations and Larger Worlds

The present graph materializes all $N^2$ edges. This is inexpensive for $N = 6$, but it will dominate at large object counts. Learned top-$k$ neighborhoods, sparse gating, locality constraints, low-rank edge factors, and hierarchical object groups could reduce the interaction cost toward $\mathcal{O}(Nk)$ while retaining source–target asymmetry. Sparsity also offers a route to interpretability, provided edge selection is evaluated against interventions rather than only visualized.

### 7.5 Multi-Step and Stochastic Futures

The current transition is deterministic and trained for one step. Repeated rollout can accumulate error:

$$\widehat{\boldsymbol{Z}}^{(t+h)} = \text{Phys}_\theta^{(h)}(\widehat{\boldsymbol{Z}}^{(t)}, \ldots), \tag{50}$$

and many real systems admit multiple valid futures. Future models should use multi-step closed-loop losses, curriculum over rollout horizon, stochastic latent variables, and uncertainty-aware energies. Conservation penalties, equivariance, or Hamiltonian structure may be introduced when the domain supplies corresponding physical semantics. For code execution, deterministic transitions are appropriate, but external input, concurrency, and undefined behavior require explicit uncertainty.

### 7.6 Required Experimental Program

A credible follow-up study should include:

- at least three to five random seeds with confidence intervals and complete per-epoch logs;
- train/validation/OOD split definitions, candidate counts, checkpoint rules, and vocabulary-fitting protocol;
- matched Transformer, recurrent, state-space, graph-network, MLP, and identity baselines at similar parameter and compute budgets;
- ablations of the importance gate, source–target separation, diagonal mask, residual update, near-zero initialization, delta targets, forbidden-fragment filter, and target-length bound;
- latency, throughput, FLOP estimates, peak activation memory, and energy use on the same Tesla T4;
- marker-controlled evaluations that separate physical correctness from serialization cues;
- interventions and counterfactuals for testing whether directed edges track causal influence rather than correlation;
- multi-step rollouts and evaluation on real program traces or physical simulators.

## 8 Conclusion

CWM is a compact experiment in changing the unit of prediction. Instead of asking a model to narrate a future one token at a time, it compresses an observation into object slots, infers ordered interactions, and advances all objects through one latent transition. The architecture removes token–token quadratic attention from the encoder and eliminates autoregressive sequential depth for a single latent forecast, while retaining a deliberate $N^2$ interaction computation over a small object set.

The high-dynamics dataset addresses a concrete optimization failure: when successor strings are near-copies of their inputs, a near-identity transition can obtain deceptively low loss. Compact delta targets and strict static-fragment filtering create a stronger learning signal. The reported Tesla T4 run converges to low latent energy, and the OOD examination assigns lower energy to directed destruction than to stasis or chaos. This is promising evidence that the model has escaped the observed latent-freezing basin.

The result remains preliminary. Directed neural edges are not yet identified causal relations; a positive-pair MSE critic can still collapse; delta-only targets can create format shortcuts; and six synthetic templates do not constitute a general theory of world dynamics. The most consequential next steps are semantic tokenization, real execution or simulator data, format-matched negative energies, permutation-aware object supervision, multi-step rollouts, sparse graphs, controlled baselines, and interventional evaluation. Under those tests, CWM can be assessed not merely as an efficient architecture, but as a candidate substrate for structured predictive reasoning on low-resource hardware.

## A Reference Forward Algorithm

For clarity, the complete training computation can be summarized as follows:

1. Embed and locally convolve T0 tokens.
2. Compute gated token-to-slot weights and pool to $\boldsymbol{Z}^{(0)} \in \mathbb{R}^{B\times N\times D}$.
3. Construct every ordered pair $(i, j)$ from separate source and target projections; map it to $\boldsymbol{E}_{ij}^{(0)} \in \mathbb{R}^{E}$.
4. Aggregate edge-conditioned messages and obtain graph-aware objects $\boldsymbol{Z}'^{(0)}$.
5. Project edges to latent forces, remove diagonal forces, and separately average outgoing and incoming effects.
6. Predict $\Delta\boldsymbol{Z}$ and form $\widehat{\boldsymbol{Z}}^{(1)} = \boldsymbol{Z}'^{(0)} + \Delta\boldsymbol{Z}$.
7. Encode the observed T1 delta with the shared object encoder.
8. Average per-object latent energy and backpropagate through all modules.

# B Tensor Shapes and Parameter Budget

Table 4: Principal tensor shapes.

| Tensor | Shape | Meaning |
|---|---|---|
| $\boldsymbol{X}$ | $[B, T]$ | Token IDs |
| $\boldsymbol{H}$ | $[B, T, D]$ | Contextual token states |
| $\boldsymbol{\Pi}$ | $[B, T, N]$ | Token-to-slot assignments |
| $\boldsymbol{Z}$ | $[B, N, D]$ | Macro-object slots |
| $\boldsymbol{P}$ | $[B, N, N, 4P]$ | Ordered pair features |
| $\boldsymbol{E}$ | $[B, N, N, E]$ | Directed edge vectors |
| $\boldsymbol{Z}'$ | $[B, N, D]$ | Graph-aware objects |
| $\boldsymbol{R}$ | $[B, N, N, D]$ | Edge forces |
| $\widehat{\boldsymbol{Z}}^{(1)}$ | $[B, N, D]$ | Predicted successor |
| $\boldsymbol{\varepsilon}$ | $[B, N]$ | Per-object energies |
| $\mathcal{L}$ | scalar | Mean training energy |

For $V = 114$, $L = 512$, $N = 6$, $D = P = 128$, $E = 64$, edge hidden width 128, and physics hidden width 512, the trainable parameter counts are

$$\#\,\text{Enc} = 197{,}505, \tag{51}$$
$$\#\,\text{Graph} = 230{,}657, \tag{52}$$
$$\#\,\text{Phys} = 567{,}424, \tag{53}$$
$$\#\,\text{Critic} = 0, \tag{54}$$
$$\#\text{Total} = 995{,}586. \tag{55}$$

The count changes linearly with vocabulary size, maximum positional-embedding length, and slot count through their embedding or query tables; activation memory changes more substantially through $T$ and $N^2$.

Table 5: Status of evidence in the present technical report.

| Claim or artifact | Current status |
|---|---|
| Executable four-layer PyTorch architecture | Implemented and smoke-tested for forward, backward, and nonzero gradient flow. |
| Synthetic generator with 10,000 examples | Implemented with deterministic seed, six balanced families, and structural validation. |
| No static examples admitted by validator | Verified syntactically for the generated corpus; not a semantic proof. |
| Training endpoint 0.000004 at epoch 30 | Reported single-run measurement; raw log and repeated-seed statistics should accompany submission. |
| OOD energy ordering | Reported point estimates for destruction, stasis, and chaos; candidate count and distribution are not yet documented. |
| Constant-depth latent transition | Established by architecture. |
| Arithmetic $\mathcal{O}(1)$ inference | Not claimed; false when $T$, $N$, $D$, or rollout horizon vary. |
| Causal identification | Not established; requires interventions and causal metrics. |
| Superiority to Transformer baselines | Not established; matched baselines are future work. |

# C Reproducibility and Claim Checklist

# References


[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is All You Need. In *Advances in Neural Information Processing Systems*, 2017.

[2] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *Advances in Neural Information Processing Systems*, 2022.

[3] L. Ribar, I. Chelombiev, L. Hudlass-Galley, C. Blake, C. Luschi, and D. Orr. SparQ Attention: Bandwidth-Efficient LLM Inference. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.

[4] J. Hoffmann, S. Borgeaud, A. Mensch, et al. Training Compute-Optimal Large Language Models. arXiv:2203.15556, 2022.

[5] F. Locatello, D. Weissenborn, T. Unterthiner, et al. Object-Centric Learning with Slot Attention. In *Advances in Neural Information Processing Systems*, 2020.

[6] P. W. Battaglia, J. B. Hamrick, V. Bapst, et al. Relational Inductive Biases, Deep Learning, and Graph Networks. arXiv:1806.01261, 2018.

[7] T. Kipf, E. Fetaya, K.-C. Wang, M. Welling, and R. Zemel. Neural Relational Inference for Interacting Systems. In *Proceedings of the 35th International Conference on Machine Learning*, 2018.

[8] T. Kipf, E. van der Pol, and M. Welling. Contrastive Learning of Structured World Models. In *International Conference on Learning Representations*, 2020.

[9] D. Ha and J. Schmidhuber. World Models. arXiv:1803.10122, 2018.

[10] Y. LeCun, S. Chopra, R. Hadsell, M. A. Ranzato, and F.-J. Huang. A Tutorial on Energy-Based Learning. In G. Bakir et al., editors, *Predicting Structured Data*, MIT Press, 2006.

[11] Y. LeCun. A Path Towards Autonomous Machine Intelligence. OpenReview preprint, version 0.9.2, 2022.